UNIVERSIDAD DON BOSCO	**UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERÍA ESCUELA DE COMPUTACIÓN**	**CICLO 02-2026**
	MATERIA | **DESARROLLO DE SOFTWARE EMPRESARIAL**	**GUIA DE LABORATORIO #9**
	PRÁCTICA | **PRUEBAS UNITARIAS**	

I. OBJETIVOS

- Que el estudiante aprenda a como0 agregar pruebas unitarias a un proyecto en .NET.
- Que el estudiante aprenda a montar una aplicación en un contenedor.

II. INTRODUCCIÓN TEORICA

¿Qué son las pruebas unitarias?

Las pruebas unitarias son un tipo de prueba de software que se centra en verificar el funcionamiento correcto de una unidad individual de código, como una función, un método o una clase. La idea es comprobar que cada unidad de código realiza las tareas para las que fue diseñada y produce los resultados esperados bajo diversas condiciones.

Ventajas de las pruebas unitarias

- **Detección temprana de errores**: permiten identificar errores en las primeras etapas del desarrollo, lo que facilita la corrección y evita problemas en fases posteriores.
- **Facilitan el Refactorizado**: al tener un conjunto de pruebas unitarias, puedes refactorizar el código con mayor confianza, sabiendo que las pruebas te ayudarán a asegurar que los cambios no introduzcan errores inesperados.
- **Documentación del código**: las pruebas unitarias sirven como una forma de documentación del comportamiento esperado del código. Pueden ayudar a otros desarrolladores a entender cómo debería funcionar una unidad específica del sistema.
- **Facilitan el desarrollo ágil**: son fundamentales en prácticas de desarrollo ágil como TDD (Test-Driven Development), donde se escriben las pruebas antes de implementar la funcionalidad.
- **Mejoran la calidad del código**: al enfocarse en unidades individuales, las pruebas unitarias fomentan la escritura de código más modular y menos dependiente, lo que puede llevar a un diseño más limpio y mantenible.
- **Reducción de costos**: aunque escribir pruebas unitarias puede requerir tiempo inicial, a largo plazo pueden reducir el costo total de desarrollo al minimizar el tiempo necesario para encontrar y corregir errores.

¿Por qué incluirlas en el desarrollo de software?

- **Prevención de regresiones**: las pruebas unitarias aseguran que los cambios en el código no rompan funcionalidades existentes. Esto es crucial para mantener la estabilidad del software a medida que evoluciona.
- **Mejoras continuas**: con pruebas unitarias, puedes mejorar continuamente el código sin miedo a introducir nuevos errores. Esto es especialmente útil en proyectos de largo plazo.
- **Mayor confianza en el código**: proporcionan una base sólida para verificar que el código funciona como se espera, lo que aumenta la confianza en la calidad del software entregado.
- **Automatización de pruebas**: las pruebas unitarias son fáciles de automatizar, lo que permite realizar pruebas frecuentes y consistentes sin intervención manual. Esto es especialmente útil en integración continua (CI) y despliegues continuos (CD).

- **Facilitan el desarrollo colaborativo**: en equipos grandes, las pruebas unitarias ayudan a garantizar que las diferentes partes del software funcionen juntas como se espera, lo que facilita el trabajo en equipo y la integración de componentes.

Test-Driven Development (TDD)

El Test-Driven Development (TDD) es una metodología de desarrollo de software que se centra en escribir pruebas antes de escribir el código funcional. Es una técnica que ayuda a garantizar que el software cumple con los requisitos desde el principio y facilita la detección de errores de manera temprana en el ciclo de desarrollo.

El proceso de TDD generalmente sigue estos pasos:

- **Escribir una prueba**: Antes de escribir el código de la funcionalidad, se crea una prueba automatizada que define cómo debe comportarse la función o el componente que se va a desarrollar. Esta prueba inicialmente fallará porque la funcionalidad aún no está implementada.
- **Escribir el código mínimo necesario para pasar la prueba**: se implementa el código más simple y mínimo que permita que la prueba pase. El enfoque está en escribir solo lo necesario para que la prueba tenga éxito.
- **Refactorizar**: una vez que la prueba pasa, se mejora o refactoriza el código para hacerlo más limpio y eficiente, manteniendo todas las pruebas funcionando correctamente.
- **Repetir**: se repite el ciclo creando nuevas pruebas para nuevas funcionalidades, escribiendo el código correspondiente y refactorizando según sea necesario.

Estructura de una prueba unitaria

- **Arrange**: establecer condiciones iniciales, preparando el escenario, declarar variables e instancias objetos (de ser necesario).
- **Act**: ejecución del fragmento de código de la prueba y del fragmento a testear.
- **Assert**: comprobación del resultado obtenido.

Buenas prácticas en las pruebas unitarias

- No se debe de probar más de una cosa en una sola prueba, es mejor hacer un test por validación.
- Deben de ser de rápida ejecución.
- Resultados consistentes.
- Deben de ser aisladas, no depender de otras pruebas u otros agentes externos.
- El fallo de una prueba debe de ser fácilmente reconocible.
- Sencillas y ágiles.
- Veracidad en las pruebas, muchas veces se logra induciendo un fallo.

III. DESARROLLO DE LA PRÁCTICA

Parte 1 – Proyecto de pruebas unitarias

1. Para esta práctica utilice el proyecto **LibrosAPI** de la **guía no. 7.**

3. En el proyecto **LibrosAPI** agregue el paquete **Microsoft.EntityFrameworkCore.InMemory** para manejar base de datos en memoria.

4. Modifique el archivo **Program.cs** del proyecto **LibrosAPI** para registrar el servicio, si ya tiene configurado SQL Server comente la línea y agregue el código que se muestra a continuación.

```
using LibrosAPI.Models;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
// builder.Services.AddDbContext<LibrosDbContext>(options =>
//     options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultCon-
nection")));   <- SQL SERVER

builder.Services.AddDbContext<LibrosDbContext>(options => options.UseInMemoryData-
base("LibrosInMemoryDb")); // <- NUEVO SERVICIO
```

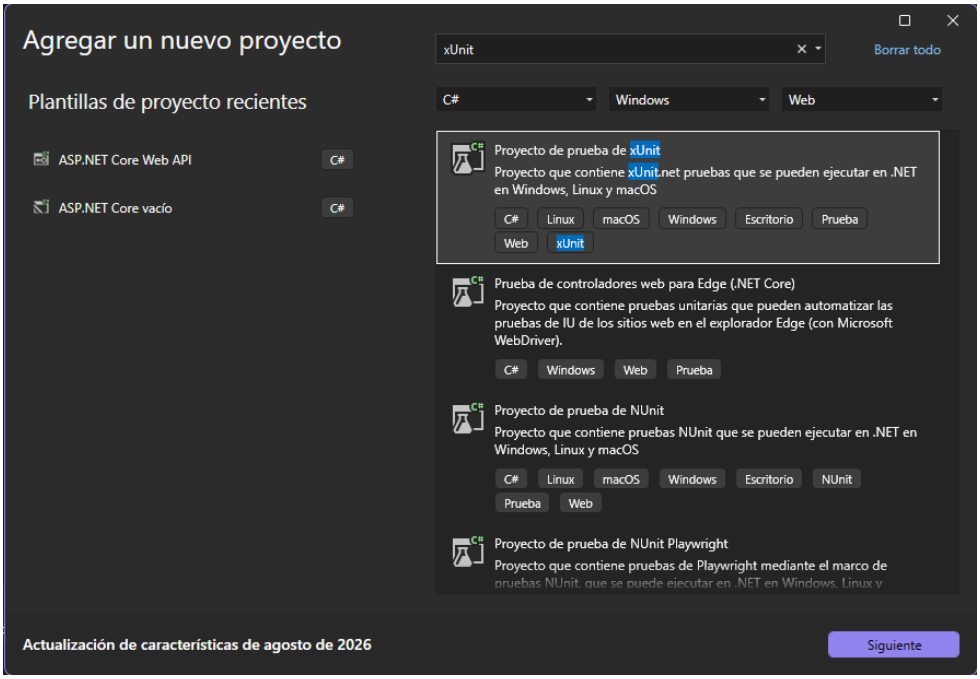
6. Modifique el controlador **LibrosController** para agregar la siguiente validación en el método **PostLibros()**.

```
[HttpPost]
public async Task<ActionResult<Libro>> PostLibro(Libro libro)
{
    // Nueva validación
    if (string.IsNullOrEmpty(libro.Titulo))
    {
        return BadRequest("El libro no tiene título.");
    }

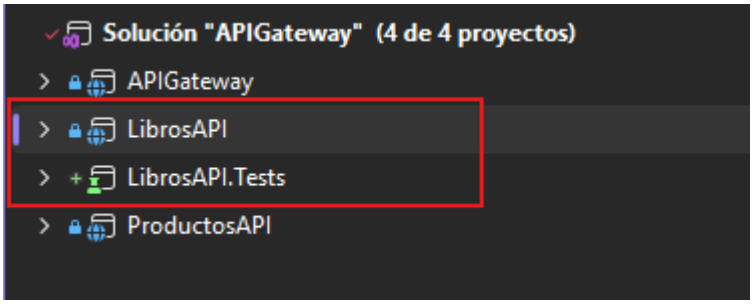
    _context.Libros.Add(libro);
    await _context.SaveChangesAsync();

    return CreatedAtAction("GetLibro", new { id = libro.Id }, libro);
}
```

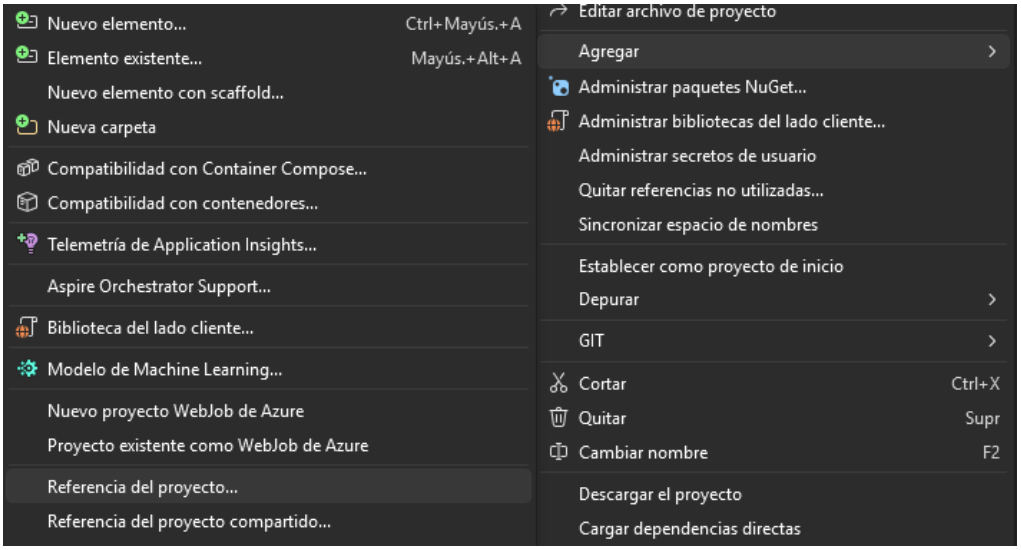
5. Agregue un nuevo proyecto de tipo **Proyecto de pruebas xUnit** con el nombre **LibrosAPI.Tests**.

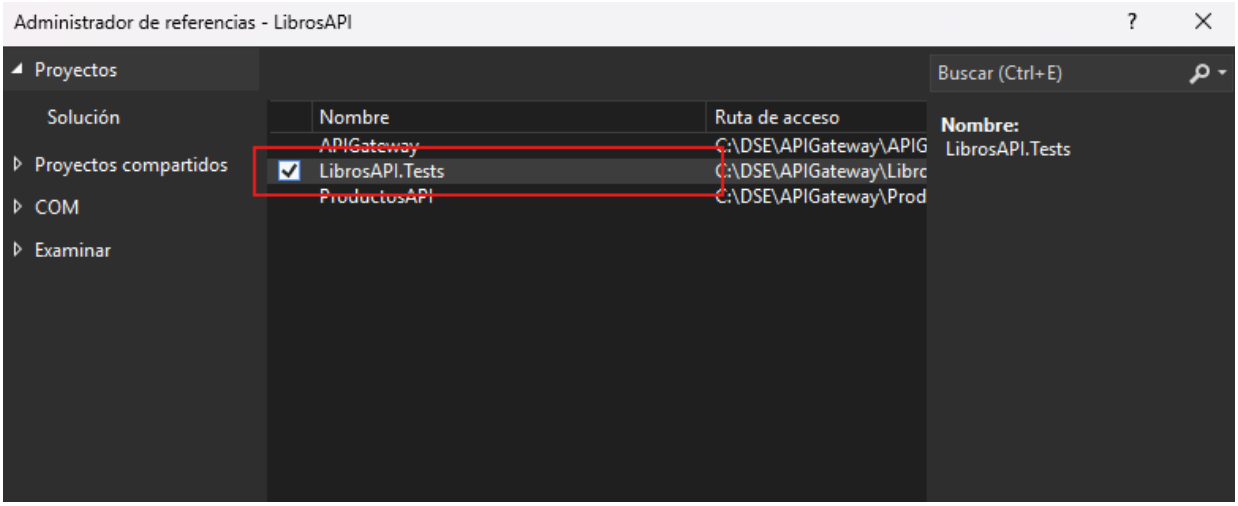


6. La solución completa debe verse como se muestra a continuación.



7. En el proyecto **LibrosAPI** agregue la referencia al proyecto **LibrosAPI.Test**, haciendo clic sobre **LibrosAPI** y seleccionando **Agregar referencia del proyecto**.





8. En el proyecto **LibrosAPI.Test** agregue la clase **Setup** con el siguiente código.

```
using LibrosAPI.Models;
using Microsoft.EntityFrameworkCore;

namespace LibrosAPI.Tests;

public class Setup
{
    public static LibrosDbContext GetDatabaseContext()
    {
        var options = new DbContextOptionsBuilder<LibrosDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .Options;

        var context = new LibrosDbContext(options);
        context.Database.EnsureCreated();
        return context;
    }
}
```

- Este código crea un contexto de base de datos en memoria para LibrosDbContext. Genera un nombre único para la base de datos, crea el contexto y verifica que la base de datos exista. Se utiliza principalmente para realizar pruebas sin necesidad de conectarse a una base de datos real.

9. En el proyecto **LibrosAPI.Tests** agregue una nueva clase **LibrosControllerTest** con el código como se muestra a continuación.

```
using LibrosAPI.Controllers;
using LibrosAPI.Models;
using Microsoft.AspNetCore.Mvc;

namespace LibrosAPI.Tests
{
    public class LibrosControllerTest
    {
        [Fact]
        public async Task PostLibro_AgregarLibro_CuandoLibroEsValido()
        {
            // Arrange
            var context = Setup.GetDatabaseContext();
```

```
            var controller = new LibrosController(context);
            var nuevoLibro = new Libro
            {
                Titulo = "Libro de prueba",
                Autor = "Autor de prueba",
                AnioPublicacion = DateTime.Now.Year
            };

            // Act
            var result = await controller.PostLibro(nuevoLibro);

            // Assert
            var createdResult = Assert.IsType<CreatedAtActionResult>(result.Re-
sult);
            var libro = Assert.IsType<Libro>(createdResult.Value);
            Assert.Equal("Libro de prueba", libro.Titulo);
        }
    }
}
```

- Este código realiza una prueba para comprobar que un libro se pueda crear correctamente.
- El atributo **[Fact]** indica que el método es una prueba unitaria en xUnit. Esto permite que el método sea reconocido y ejecutado automáticamente por el sistema de pruebas para comprobar que el código funciona correctamente.
- En la sección **Arrange** se prepara el contexto de la base de datos, el controlador y un nuevo libro con sus datos.
- En **Act** se ejecuta el método **PostLibro** para registrar el libro.
- Finalmente, en **Assert** se verifica que el resultado sea una creación exitosa y que el título del libro guardado sea "Libro de prueba".

10. Agregue los siguientes casos de prueba al controlador **LibrosControllerTest**.

```
[Fact]
public async Task GetLibro_RetornaLibro_CuandoIdEsValido()
{
    // Arrange
    var context = Setup.GetDatabaseContext();
    var controller = new LibrosController(context);
    var nuevoLibro = new Libro
    {
        Titulo = "Libro de prueba",
        Autor = "Autor de prueba",
        AnioPublicacion = DateTime.Now.Year
    };
    context.Libros.Add(nuevoLibro);
    await context.SaveChangesAsync();

    // Act
    var result = await controller.GetLibro(nuevoLibro.Id);

    // Assert
    var actionResult = Assert.IsType<ActionResult<Libro>>(result);
    var returnValue = Assert.IsType<Libro>(actionResult.Value);
    Assert.Equal("Libro de prueba", returnValue.Titulo);
}
```

```
[Fact]
public async Task GetLibro_RetornaNotFound_CuandoIdNoExiste()
{
    // Arrange
    var context = Setup.GetDatabaseContext();
    var controller = new LibrosController(context);

    // Act
    var result = await controller.GetLibro(999);

    // Assert
    Assert.IsType<NotFoundResult>(result.Result);
}

[Fact]
public async Task PostLibro_NoAgregarLibro_CuandoNoTieneTitulo()
{
    // Arrange
    var context = Setup.GetDatabaseContext();
    var controller = new LibrosController(context);
    var nuevoLibro = new Libro
    {
        Titulo = null,
        Autor = "Autor de prueba",
        AnioPublicacion = DateTime.Now.Year
    };

    // Act
    var result = await controller.PostLibro(nuevoLibro);

    // Assert
    Assert.IsType<BadRequestObjectResult>(result.Result);
}

[Fact]
public async Task PostLibro_IncrementaConteo_CuandoSeAgregaNuevoLibro()
{
    // Arrange
    var context = Setup.GetDatabaseContext();
    var controller = new LibrosController(context);
    var nuevoLibro1 = new Libro
    {
        Titulo = "Libro de prueba 1",
        Autor = "Autor de prueba 1",
        AnioPublicacion = DateTime.Now.Year
    };

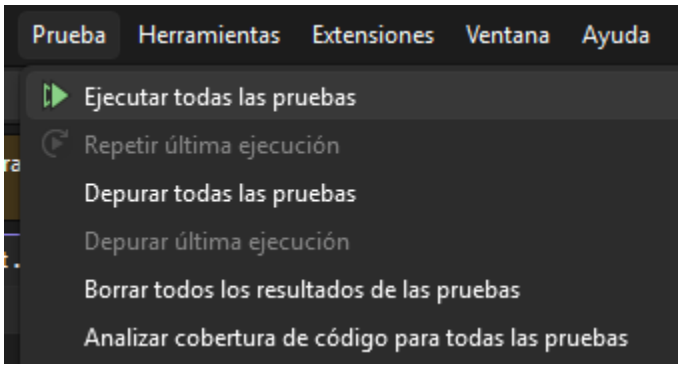
    await controller.PostLibro(nuevoLibro1);

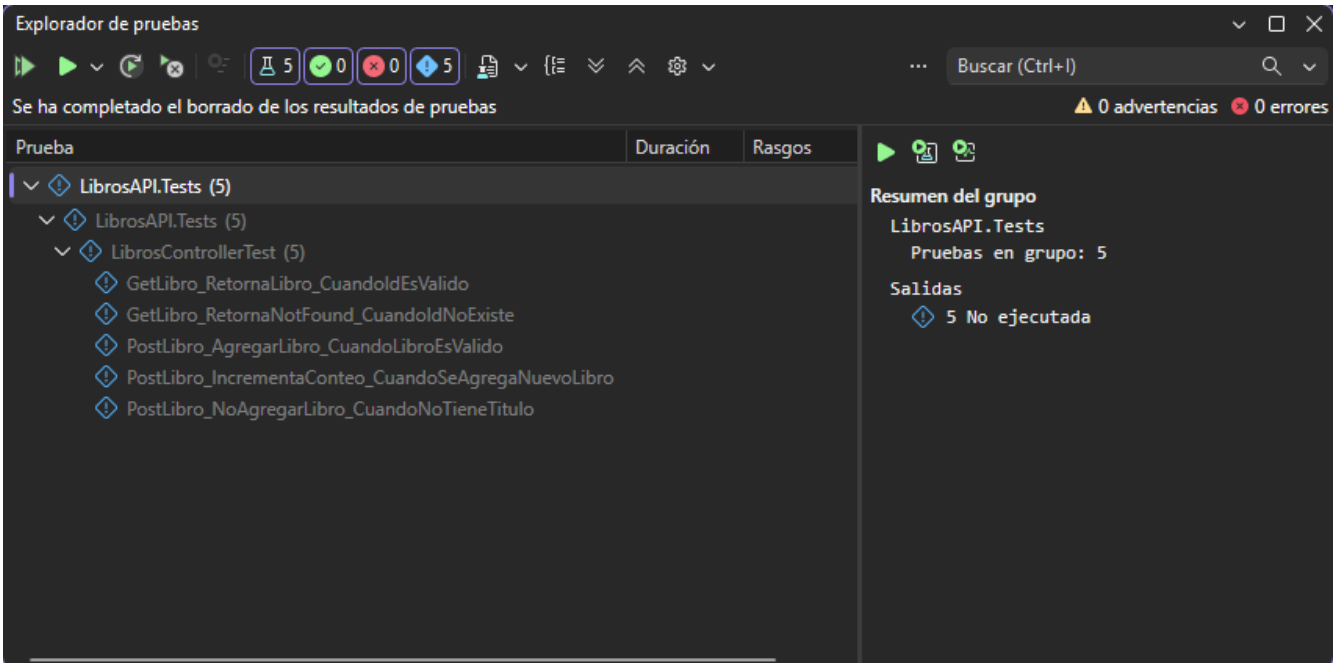
    var nuevoLibro2 = new Libro
    {
        Titulo = "Libro de prueba 2",
        Autor = "Autor de prueba 2",
        AnioPublicacion = DateTime.Now.Year
    };

    // Act
    var result = await controller.PostLibro(nuevoLibro2);
    var libros = context.Libros.ToList();
```

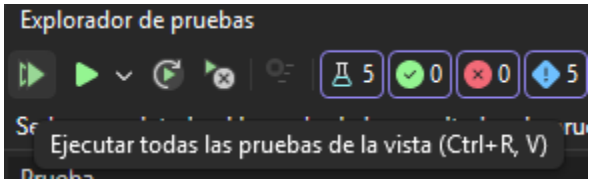
```
    // Assert
    Assert.Equal(2, libros.Count);
}
```

11. Ejecute las pruebas en **Visual Studio** haciendo clic en **Pruebas > Explorador de Pruebas**, posteriormente se mostrará una ventana con todos los archivos en los cuales se encuentren pruebas definidas.

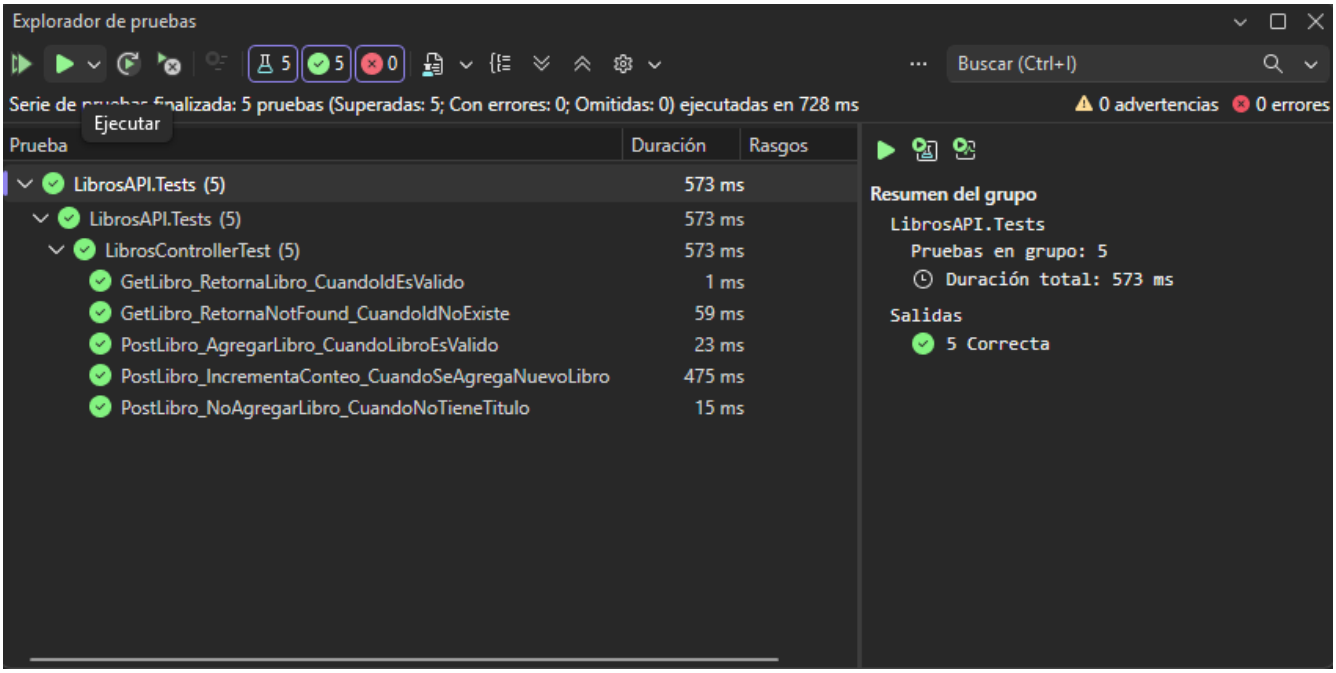




12. En esa ventana podrá ejecutar todas las pruebas o solo las que se seleccionen, para ejecutar todas haga clic en el botón **Ejecutar todas las pruebas de la vista**.



13. En la ventana resultando podrá visualizar el resumen de ejecución de las pruebas, donde ser marca en color verde las que fueron exitosas y en rojo las que no.



Las pruebas escritas se definen de la siguiente manera:

- **PostLibro_AgregaLibro_CuandoLibroEsValido**: verifica que un libro válido se agregue correctamente.
- **GetLibro_RetornaLibro_CuandoIdEsValido**: comprueba que se retorne el libro correcto cuando se busca por un ID válido.
- **GetLibro_RetornaNotFound_CuandoIdNoExiste**: verifica que se retorne un NotFound cuando se busca un libro con un ID inexistente.
- **PostLibro_NoAgregaLibro_CuandoTituloEsNulo**: asegura que no se agregue un libro si el título es nulo.
- **PostLibro_IncrementaConteo_CuandoSeAgregaNuevoLibro**: comprueba que el conteo de libros en la base de datos aumente al agregar un nuevo libro.

IV. DESARROLLO DE HABILIDADES

Ejercicio 1

Desarrolle una aplicación en ASP .NET Core para gestionar el registro de personas naturales.

Campos mínimos requeridos para Personas:

- Id
- Primer nombre
- Segundo nombre
- Primer apellido
- Segundo nombre
- DUI
- Fecha de nacimiento
-

Además, considere las siguientes validaciones:

- Primero nombre y primer apellido son requeridos.
- Segundo nombre y apellido son opcionales.
- Tanto los nombres como los apellidos no pueden exceder la longitud de 100 caracteres.
- Fecha de nacimiento no puede ser nula y debe ser una fecha valida.
- Validar el formato del DUI (01234567-8)
- Escriba las pruebas correspondientes para comprobar que el controlador escrito en la API trabaje de manera adecuada.